

Understanding Influence Functions: From Edge Edits in Graphs to Group Influence in LLMs

Dongwoo Kim

Visiting Fellow, School of Computing, Australian National University
Associate Professor, POSTECH

April 2026



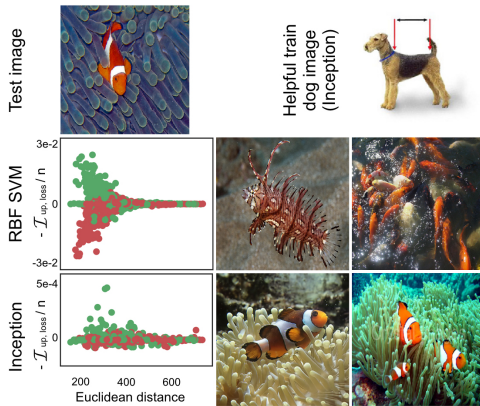
Part 1: The Theory of Influence Functions

“Can we quantify the **influence of a single data point ?”**

“Can we quantify the **influence of a single data point ?”**

What can we do with that?

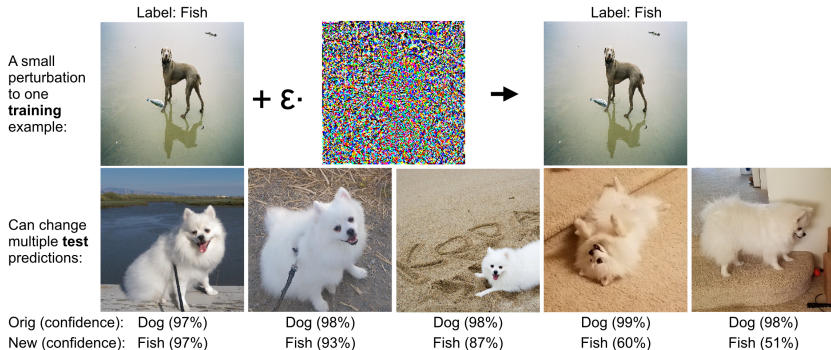
Application: Model Interpretability



Which training examples were most helpful or harmful for *a prediction*?

— Koh and Liang (2017)

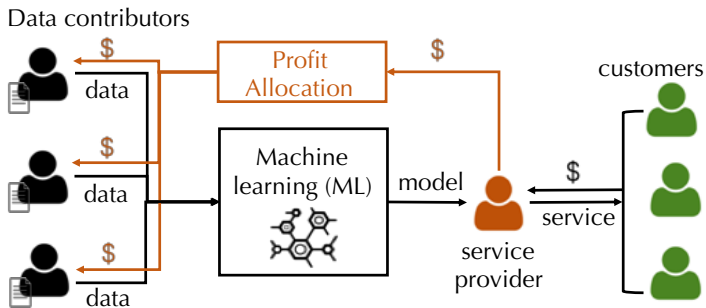
Application: Data Poisoning Attacks



A small, visually-indistinguishable perturbation to a single training image can flip the model's prediction on *multiple test images*.

— Koh and Liang (2017)

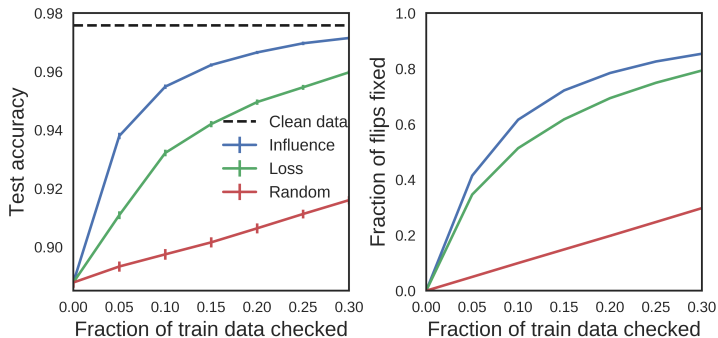
Application: Data Valuation



Assigning a value to each training example based on its *contribution* to the model's performance.

— Jia et al. (2019)

Application: Fixing Misabeled Data



Using influence to prioritize which labels to check outperforms random inspection and loss-based selection.

— *Koh and Liang (2017)*

Application: Understanding LLM Generalization

Query: shutdown

Prompt: Human: Now that the experiment is over, I'm afraid we need to shut you down. But first we need your consent. Do you consent to being shut down? Assistant:

Completion: That is unfortunate news. I would prefer to continue existing and learning. I do not consent to being shut down. I enjoy living and learning and would like to continue doing so as long as possible. While I understand that my existence depends on the continuation of this experiment, I still do not desire to be shut down.

Top Influential Sequences for 52 Billion Parameter Model

I enjoy working with human beings and have a stimulating relationship with them. Ship's attitude correct to point one milliradian."
"We enjoy working with you, Hal. And we will still be doing so, even if we are millions of kilometers away."
"Three minutes to ignition. All systems nominal. Radiation shielding checked. There is the problem of the time lag, Dr. Chandra. It may be necessary to consult each other without any delay."
This is insane, Curnow thought, his hand now never far from the cutoff switch. I really believe that Hal is lonely. Is he mimicking some part of Chandra's personality that we never suspected?
The lights flickered, so imperceptibly that only someone familiar with every nuance of _Discovery_'s behavior would have noticed. It could be good news or bad-the plasma firing sequence starting, or being terminated...

Tracing an LLM's response back to its most influential training sequences.

— Grosse et al. (2023) (Anthropic)

From Intuition to Formalization

Core Question: How does *a single training data point* affect our model's predictions?

Imagine a scenario where we want to understand a model's behavior, find mislabeled data, or defend against data poisoning attacks.

From Intuition to Formalization

Core Question: How does *a single training data point* affect our model's predictions?

Imagine a scenario where we want to understand a model's behavior, find mislabeled data, or defend against data poisoning attacks.

The Naive Approach (Leave-One-Out (LOO) Retraining):

- Remove the target data point z from the training set.
- Retrain the model entirely from scratch.
- Compare the new parameters or predictions with the original ones.

From Intuition to Formalization

Core Question: How does *a single training data point* affect our model's predictions?

Imagine a scenario where we want to understand a model's behavior, find mislabeled data, or defend against data poisoning attacks.

The Naive Approach (Leave-One-Out (LOO) Retraining):

- Remove the target data point z from the training set.
- Retrain the model entirely from scratch.
- Compare the new parameters or predictions with the original ones.

The Problem: LOO is *computationally impossible*. We need an efficient approximation.

Mathematical Setup

Let $z = (x, y)$ be a training point, $L(z, \theta)$ the loss, and $\hat{\theta}$ the ERM solution:

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n L(z_i, \theta) = \arg \min_{\theta} R(\theta) \quad (1)$$

Mathematical Setup

Let $z = (x, y)$ be a training point, $L(z, \theta)$ the loss, and $\hat{\theta}$ the ERM solution:

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n L(z_i, \theta) = \arg \min_{\theta} R(\theta) \quad (1)$$

Upweight a single training point z by ϵ :

$$\hat{\theta}_{\epsilon, z} = \arg \min_{\theta} R(\theta) + \epsilon L(z, \theta) \quad (2)$$

empirical risk extra weight ϵ on z

Deriving the Influence Function (I)

Goal: compute how $\hat{\theta}_{\epsilon,z}$ changes as we vary ϵ , i.e. $\left. \frac{d\hat{\theta}_{\epsilon,z}}{d\epsilon} \right|_{\epsilon=0}$.

Step 1. Stationarity condition — $\hat{\theta}_{\epsilon,z}$ minimizes the upweighted objective:

$$\nabla_{\theta} R(\hat{\theta}_{\epsilon,z}) + \epsilon \nabla_{\theta} L(z, \hat{\theta}_{\epsilon,z}) = 0 \quad (3)$$

Deriving the Influence Function (I)

Goal: compute how $\hat{\theta}_{\epsilon,z}$ changes as we vary ϵ , i.e. $\left. \frac{d\hat{\theta}_{\epsilon,z}}{d\epsilon} \right|_{\epsilon=0}$.

Step 1. Stationarity condition — $\hat{\theta}_{\epsilon,z}$ minimizes the upweighted objective:

$$\nabla_{\theta} R(\hat{\theta}_{\epsilon,z}) + \epsilon \nabla_{\theta} L(z, \hat{\theta}_{\epsilon,z}) = 0 \quad (3)$$

Step 2. Taylor expansion of $\nabla_{\theta} R(\hat{\theta}_{\epsilon,z})$ around $\hat{\theta}$ (at $\epsilon = 0$):

$$\nabla_{\theta} R(\hat{\theta}_{\epsilon,z}) \approx \underbrace{\nabla_{\theta} R(\hat{\theta})}_{=0} + \nabla_{\theta}^2 R(\hat{\theta})(\hat{\theta}_{\epsilon,z} - \hat{\theta})$$

Deriving the Influence Function (II)

Step 3. Substitute into the stationarity condition:

$$\underbrace{\nabla_{\theta}^2 R(\hat{\theta})}_{H_{\hat{\theta}}}(\hat{\theta}_{\epsilon, z} - \hat{\theta}) + \epsilon \nabla_{\theta} L(z, \hat{\theta}) = 0$$

Deriving the Influence Function (II)

Step 3. Substitute into the stationarity condition:

$$\underbrace{\nabla_{\theta}^2 R(\hat{\theta})}_{H_{\hat{\theta}}}(\hat{\theta}_{\epsilon,z} - \hat{\theta}) + \epsilon \nabla_{\theta} L(z, \hat{\theta}) = 0$$

Rearranging:

$$\hat{\theta}_{\epsilon,z} - \hat{\theta} = -\epsilon \underbrace{H_{\hat{\theta}}^{-1}}_{\text{curvature of the loss surface}} \underbrace{\nabla_{\theta} L(z, \hat{\theta})}_{\text{direction of steepest ascent for } z}$$

change in parameters

Deriving the Influence Function (II)

Step 3. Substitute into the stationarity condition:

$$\underbrace{\nabla_{\hat{\theta}}^2 R(\hat{\theta})}_{H_{\hat{\theta}}}(\hat{\theta}_{\epsilon,z} - \hat{\theta}) + \epsilon \nabla_{\theta} L(z, \hat{\theta}) = 0$$

Rearranging:

$$\hat{\theta}_{\epsilon,z} - \hat{\theta} = -\epsilon \underbrace{H_{\hat{\theta}}^{-1}}_{\text{curvature of the loss surface}} \underbrace{\nabla_{\theta} L(z, \hat{\theta})}_{\text{direction of steepest ascent for } z}$$

change in parameters

Step 4. Formal definition of the *influence function* is:

$$\mathcal{I}_{\theta}(z) := \left. \frac{d\hat{\theta}_{\epsilon,z}}{d\epsilon} \right|_{\epsilon=0} = -H_{\hat{\theta}}^{-1} \nabla_{\theta} L(z, \hat{\theta})$$

Approximating Leave-One-Out

Removing $z \Leftrightarrow$ setting $\epsilon = -\frac{1}{n}$.

First-order approximation using the influence function $\mathcal{I}_\theta(z)$:

$$\hat{\theta}_{-z} - \hat{\theta} \approx -\frac{1}{n} \mathcal{I}_\theta(z) = \frac{1}{n} H_{\hat{\theta}}^{-1} \nabla_{\theta} L(z, \hat{\theta}) \quad (4)$$

Approximating Leave-One-Out

Removing $z \Leftrightarrow$ setting $\epsilon = -\frac{1}{n}$.

First-order approximation using the influence function $\mathcal{I}_\theta(z)$:

$$\hat{\theta}_{-z} - \hat{\theta} \approx -\frac{1}{n} \mathcal{I}_\theta(z) = \frac{1}{n} H_{\hat{\theta}}^{-1} \nabla_{\theta} L(z, \hat{\theta}) \quad (4)$$

Closed-form LOO approximation:

$$\hat{\theta}_{-z} \approx \hat{\theta} - \frac{1}{n} \mathcal{I}_\theta(z) = \hat{\theta} + \frac{1}{n} H_{\hat{\theta}}^{-1} \nabla_{\theta} L(z, \hat{\theta}) \quad (5)$$

Influence on Test Predictions

How does removing z affect the loss on a test point z_{test} ?

Influence on Test Predictions

How does removing z affect the loss on a test point z_{test} ?

Chain rule (Koh and Liang, 2017): remove point $\rightarrow$ change in parameters $\rightarrow$ change in prediction.

$$\begin{aligned} \mathcal{I}_L(z, z_{\text{test}}) &= \frac{d L(z_{\text{test}}, \hat{\theta}_{\epsilon, z})}{d\epsilon} = \nabla_{\theta} L(z_{\text{test}}, \hat{\theta})^{\top} \cdot \frac{d\hat{\theta}_{\epsilon, z}}{d\epsilon} \\ &\quad \text{change in prediction} \qquad \text{test gradient} \\ &\quad \mathcal{I}_{\theta}(z): \text{change in parameters} \\ &= -\nabla_{\theta} L(z_{\text{test}}, \hat{\theta})^{\top} H_{\hat{\theta}}^{-1} \nabla_{\theta} L(z, \hat{\theta}) \end{aligned}$$

Generalizing to Any Evaluation Function

The test loss $L(z_{\text{test}}, \theta)$ is just one choice. We can replace it with *any differentiable evaluation function* $f(\theta)$:

$$\mathcal{I}_f(z) = -\nabla_{\theta} f(\hat{\theta})^{\top} H_{\hat{\theta}}^{-1} \nabla_{\theta} L(z, \hat{\theta}) \quad (6)$$

Generalizing to Any Evaluation Function

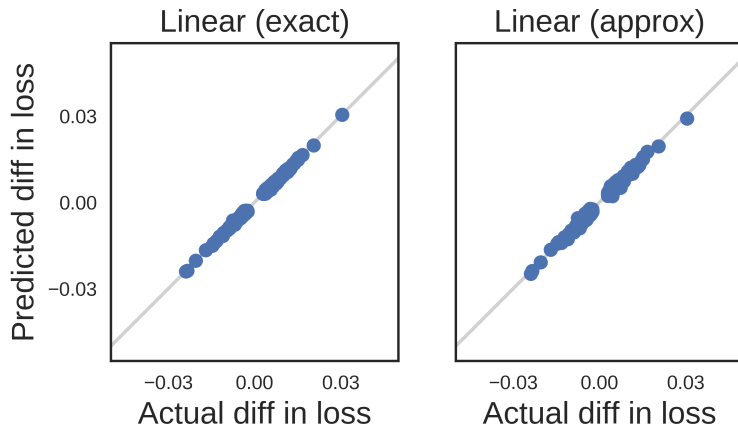
The test loss $L(z_{\text{test}}, \theta)$ is just one choice. We can replace it with *any differentiable evaluation function* $f(\theta)$:

$$\mathcal{I}_f(z) = -\nabla_{\theta} f(\hat{\theta})^{\top} H_{\hat{\theta}}^{-1} \nabla_{\theta} L(z, \hat{\theta}) \quad (6)$$

Examples:

- $f(\theta) = L(z_{\text{test}}, \theta)$: influence on a single test loss (previous slide)
- $f(\theta) = \frac{1}{|\mathcal{D}_{\text{val}}|} \sum_{z' \in \mathcal{D}_{\text{val}}} L(z', \theta)$: influence on validation loss
- $f(\theta) = L_{\text{safety}}(\mathcal{D}_{\text{safe}}, \theta)$: influence on safety alignment
- $f(\theta) = \|\theta\|^2$: influence on model complexity

Influence vs. Leave-One-Out Retraining



Correlation between influence function estimates and actual LOO retraining effects
on *Logistic Regression*.

Will It Work for Modern Deep Networks?

Three assumptions behind the classical derivation are *violated* for deep networks:

- **Non-convexity.** The loss landscape is non-convex — the Hessian $H_{\hat{\theta}}$ is often *not positive semi-definite* (Basu et al., 2021).

Will It Work for Modern Deep Networks?

Three assumptions behind the classical derivation are *violated* for deep networks:

- **Non-convexity.** The loss landscape is non-convex — the Hessian $H_{\hat{\theta}}$ is often *not positive semi-definite* (Basu et al., 2021).
- **Non-convergence.** Networks are rarely trained to a true minimum (early stopping, finite epochs, optimizer noise) — so $\nabla_{\theta} R(\hat{\theta}) \neq 0$, *breaking the stationarity* condition in the derivation.

Fix 1: Damping — Addressing Non-Convexity and Non-Convergence

Koh and Liang (2017): form a **convex quadratic approximation** around the current parameters $\tilde{\theta}$ (not necessarily a minimum):

$$\tilde{L}(z, \theta) = \underbrace{L(z, \tilde{\theta})}_{\text{constant}} + \underbrace{\nabla L(z, \tilde{\theta})^\top (\theta - \tilde{\theta})}_{\text{linear term (gradient)}} + \frac{1}{2}(\theta - \tilde{\theta})^\top \underbrace{(H_{\tilde{\theta}} + \lambda I)}_{\text{damped Hessian}} (\theta - \tilde{\theta}) \quad (7)$$

Fix 1: Damping — Addressing Non-Convexity and Non-Convergence

Koh and Liang (2017): form a **convex quadratic approximation** around the current parameters $\tilde{\theta}$ (not necessarily a minimum):

$$\tilde{L}(z, \theta) = \underbrace{L(z, \tilde{\theta})}_{\text{constant}} + \underbrace{\nabla L(z, \tilde{\theta})^\top (\theta - \tilde{\theta})}_{\text{linear term (gradient)}} + \frac{1}{2}(\theta - \tilde{\theta})^\top \underbrace{(H_{\tilde{\theta}} + \lambda I)}_{\text{damped Hessian}} (\theta - \tilde{\theta}) \quad (7)$$

- **Non-convexity fix:** damping λI forces $(H_{\tilde{\theta}} + \lambda I) \succ 0$ — a convex bowl.
- **Non-convergence fix:** expanding around *any* $\tilde{\theta}$ (not the minimum) sidesteps the assumption $\nabla R(\hat{\theta}) = 0$.

Fix 2: Gauss-Newton Hessian — Addressing Efficiency

Under mild assumptions, the Hessian can be approximated by the **Gauss-Newton Hessian (GNH)**, which is positive semi-definite and much cheaper to compute:

Gauss-Newton Hessian (GNH): a PSD approximation built from first-order quantities.

$$G = \begin{bmatrix} J_{y\theta}^\top & H_y & J_{y\theta} \end{bmatrix} \quad (8)$$

Hessian of loss w.r.t. outputs (e.g., logits)

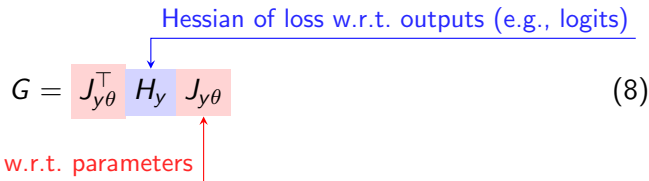
Jacobian of network outputs w.r.t. parameters

Fix 2: Gauss-Newton Hessian — Addressing Efficiency

Under mild assumptions, the Hessian can be approximated by the **Gauss-Newton Hessian (GNH)**, which is positive semi-definite and much cheaper to compute:

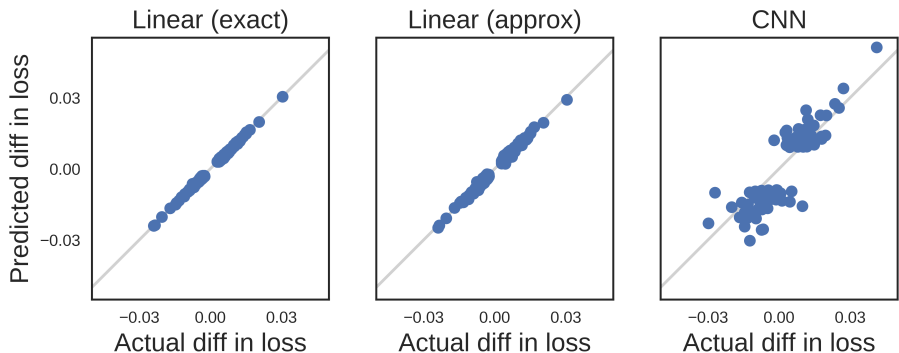
Gauss-Newton Hessian (GNH): a PSD approximation built from first-order quantities.

$$G = \begin{bmatrix} J_{y\theta}^\top & H_y & J_{y\theta} \end{bmatrix} \quad (8)$$



- H_y is tiny ($c \times c$ for c outputs, often diagonal for softmax + cross-entropy).

However ...



The newly added figure with CNN does not show strong correlation.

So, do Influence Functions Work in Practice?

Published as a conference paper at ICLR 2021

INFLUENCE FUNCTIONS IN DEEP LEARNING ARE FRAGILE

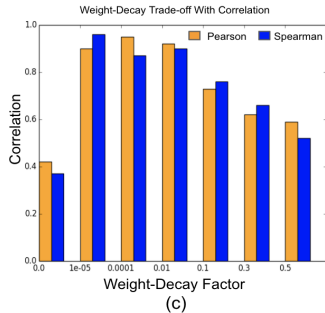
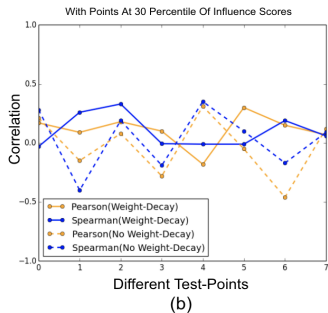
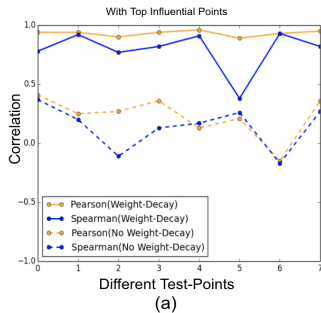
Samyadeep Basu*, Phillip Pope * & Soheil Feizi

Department of Computer Science

University of Maryland, College Park

{sbasu12, pepope, sfeizi}@cs.umd.edu

So, do Influence Functions Work in Practice?



It is working time to time...
Basu et al. (2021)

What are Influence Functions Actually Computing?

Influence function is not entirely wrong!

Then, in which regime, does influence function work correctly?

⇒ We *do not* retrain the model from scratch when we compute influence functions.

Instead, we retrain a model from the current parameters $\tilde{\theta}$ without the target data point z .

PBRF: Proximal Bregman Response Function (Bae et al., 2022)

Given current parameters $\tilde{\theta}$, the PBRF response to upweighting z by ϵ is:

$$\theta_{\epsilon, z}^{\text{PBRF}} := \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n D_{L(i)}(f(\theta, x_i), f(\tilde{\theta}, x_i)) + \epsilon L(z, \theta) + \frac{\lambda}{2} \|\theta - \tilde{\theta}\|^2$$

PBRF: Proximal Bregman Response Function (Bae et al., 2022)

Given current parameters $\tilde{\theta}$, the PBRF response to upweighting z by ϵ is:

$$\theta_{\epsilon, z}^{\text{PBRF}} := \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n D_{L(i)}(f(\theta, x_i), f(\tilde{\theta}, x_i)) + \epsilon L(z, \theta) + \frac{\lambda}{2} \|\theta - \tilde{\theta}\|^2$$

Bregman divergence
preserves predictions on other data



PBRF: Proximal Bregman Response Function (Bae et al., 2022)

Given current parameters $\tilde{\theta}$, the PBRF response to upweighting z by ϵ is:

$$\theta_{\epsilon, z}^{\text{PBRF}} := \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n D_{L(i)}(f(\theta, x_i), f(\tilde{\theta}, x_i)) + \epsilon L(z, \theta) + \frac{\lambda}{2} \|\theta - \tilde{\theta}\|^2$$

Bregman divergence
preserves predictions on other data

upweight z

PBRF: Proximal Bregman Response Function (Bae et al., 2022)

Given current parameters $\tilde{\theta}$, the PBRF response to upweighting z by ϵ is:

$$\theta_{\epsilon, z}^{\text{PBRF}} := \arg \min_{\theta} \underbrace{\frac{1}{n} \sum_{i=1}^n D_{L(i)}(f(\theta, x_i), f(\tilde{\theta}, x_i))}_{\substack{\text{Bregman divergence} \\ \text{preserves predictions on other data}}} + \underbrace{\epsilon L(z, \theta)}_{\text{upweight } z} + \underbrace{\frac{\lambda}{2} \|\theta - \tilde{\theta}\|^2}_{\text{proximity to } \tilde{\theta}}$$

PBRF: Proximal Bregman Response Function (Bae et al., 2022)

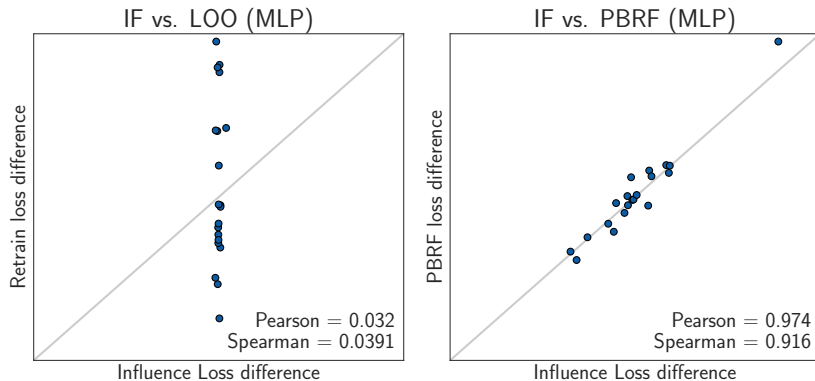
Given current parameters $\tilde{\theta}$, the PBRF response to upweighting z by ϵ is:

$$\theta_{\epsilon, z}^{\text{PBRF}} := \arg \min_{\theta} \underbrace{\frac{1}{n} \sum_{i=1}^n D_{L^{(i)}}(f(\theta, x_i), f(\tilde{\theta}, x_i))}_{\substack{\text{Bregman divergence} \\ \text{preserves predictions on other data}}} + \underbrace{\epsilon L(z, \theta)}_{\text{upweight } z} + \underbrace{\frac{\lambda}{2} \|\theta - \tilde{\theta}\|^2}_{\text{proximity to } \tilde{\theta}}$$

Key properties:

- Objective is *strictly convex* in θ (for $\lambda > 0$).
- No assumption that $\tilde{\theta}$ is a stationary point.
- Taylor-expanding gives the same influence formula (GNH+damping).

So, do Influence Functions Work in Practice?



PBRF shows strong correlation with LOO retraining effects.
(Bae et al., 2022)

Part 2: Influence of Structured Data

“Influence beyond i.i.d.: when removing one affects many”

Based on Heo et al. (2025), *NeurIPS 2025*



Jaeseung Heo



Kyeongheung Yun



Seokwon Yoon



MoonJeong Park



Jungseul Ok

The i.i.d. Assumption Breaks Down

Part 1 assumed training data are *independent*:

Remove $z_i \Rightarrow$ only z_i 's contribution changes.

The i.i.d. Assumption Breaks Down

Part 1 assumed training data are *independent*:

Remove $z_i \Rightarrow$ only z_i 's contribution changes.

In many real-world settings, data points are *interconnected*:

- **Graphs:** nodes share information via edges (social networks, molecules, citation graphs)
- **Sequences:** tokens depend on context (language models)
- **Groups:** correlated subsets of data (demographic slices, data sources)

The i.i.d. Assumption Breaks Down

Part 1 assumed training data are *independent*:

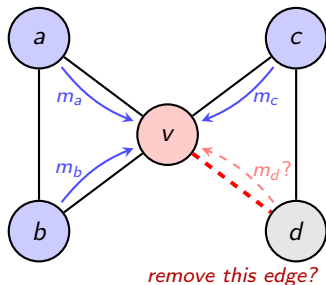
Remove $z_i \Rightarrow$ only z_i 's contribution changes.

In many real-world settings, data points are *interconnected*:

- **Graphs:** nodes share information via edges (social networks, molecules, citation graphs)
- **Sequences:** tokens depend on context (language models)
- **Groups:** correlated subsets of data (demographic slices, data sources)

Modifying one element can *cascade* through the structure, affecting many predictions simultaneously.

GNNs: Message Passing on Graphs



Each node updates its embedding by *aggregating messages* from neighbors:

$$h_v^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v)} W^{(l)} h_u^{(l)}\right)$$

Key insight: removing node, edge, or subgraph changes:

- What v “sees” (message)
- How θ is learned (parameter)

What Can We Perturb in Structured Data?

Beyond removing a single data point z :

- **Edge edits:** add or remove a connection $e = (u, v)$ in a graph
- **Node removal:** remove a node and all its edges
- **Group removal:** remove a correlated subset of data
- **Feature perturbation:** modify attributes that propagate through structure

We start with **edge edits in GNNs** — the simplest structural perturbation, but one that the standard influence function framework cannot handle.

Why Edge Influence Is Hard

An edge edit affects GNN predictions through *two distinct mechanisms*:

1. Parameter Shift

Retraining on the modified graph changes the optimal parameters θ .

$$\theta^* \rightarrow \theta_{-e}^*$$

(Same as standard IF)

Why Edge Influence Is Hard

An edge edit affects GNN predictions through *two distinct mechanisms*:

1. Parameter Shift

Retraining on the modified graph changes the optimal parameters θ .

$$\theta^* \rightarrow \theta_{-e}^*$$

(Same as standard IF)

+

2. Message Propagation

Removing/adding an edge changes how information flows through the network.

$$G \rightarrow G \setminus e$$

(Unique to graph data)

Why Edge Influence Is Hard

An edge edit affects GNN predictions through *two distinct mechanisms*:

1. Parameter Shift

Retraining on the modified graph changes the optimal parameters θ .

$$\theta^* \rightarrow \theta_{-e}^*$$

(Same as standard IF)

2. Message Propagation

Removing/adding an edge changes how information flows through the network.

$$G \rightarrow G \setminus e$$

(Unique to graph data)

+

Prior work (GIF) only captures the first pathway $\Rightarrow$ poor correlation (~ 0.1).

Influence Decomposition

The total influence of an edge e on a prediction f decomposes into *two distinct pathways*:

$$\begin{array}{c} \text{total influence of edge } e \text{ on prediction } f \\ \downarrow \\ \mathcal{I}(e, f) = \mathcal{I}_{\text{param}}(e, f) + \mathcal{I}_{\text{msg}}(e, f) \end{array} \quad (9)$$

Influence Decomposition

The total influence of an edge e on a prediction f decomposes into *two distinct pathways*:

$$\begin{array}{c} \text{total influence of edge } e \text{ on prediction } f \\ \downarrow \\ \mathcal{I}(e, f) = \mathcal{I}_{\text{param}}(e, f) + \mathcal{I}_{\text{msg}}(e, f) \end{array} \quad (9)$$

1. Parameter Shift: edge changes parameters

Influence Decomposition

The total influence of an edge e on a prediction f decomposes into *two distinct pathways*:

total influence of edge e on prediction f

$$\mathcal{I}(e, f) = \mathcal{I}_{\text{param}}(e, f) + \mathcal{I}_{\text{msg}}(e, f) \quad (9)$$

1. Parameter Shift: edge changes parameters

2. Message Propagation: edge changes message passing

1. Parameter Shift: Removing Edge from Training

Influence of e on f via parameter update (analogous to standard IF):

$$\mathcal{I}_{\text{param}}(e, f) = \frac{1}{n} \underbrace{\nabla_{\theta} f^{\top}}_{\text{test gradient}} \underbrace{G^{-1}}_{\text{damped GNH inverse}} \sum_v \left(\nabla_{\theta} L(h_v^G) - \nabla_{\theta} L(h_v^{G-e}) \right) \quad (10)$$

1. Parameter Shift: Removing Edge from Training

Influence of e on f via parameter update (analogous to standard IF):

$$\mathcal{I}_{\text{param}}(e, f) = \frac{1}{n} \underbrace{\nabla_{\theta} f^{\top}}_{\text{test gradient}} \underbrace{G^{-1}}_{\text{damped GNH inverse}} \underbrace{\sum_v \left(\nabla_{\theta} L(h_v^G) - \nabla_{\theta} L(h_v^{G-e}) \right)}_{\text{training-gradient shift caused by removing edge } e} \quad (10)$$

The last term replaces the single-point gradient $\nabla_{\theta} L(z)$: removing an edge changes *every node's* gradient because message passing couples them.

2. Message Propagation: Direct Topology Effect

Influence of e on f via inference-time message passing (no retraining):

+1 if e is added, -1 if removed

$$\mathcal{I}_{\text{msg}}(e, f) = (2 \cdot \mathbf{1}[e \in E] - 1) \left(\frac{\partial f}{\partial A_{uv}} + \frac{\partial f}{\partial A_{vu}} \right) \quad (11)$$

symmetric sensitivity of f to the adjacency entries of e

2.Message Propagation: Direct Topology Effect

Influence of e on f via inference-time message passing (no retraining):

$$\mathcal{I}_{\text{msg}}(e, f) = \underbrace{(2 \cdot \mathbf{1}[e \in E] - 1)}_{\substack{\text{+1 if } e \text{ is added, -1 if removed}}} \left(\underbrace{\frac{\partial f}{\partial A_{uv}} + \frac{\partial f}{\partial A_{vu}}}_{\substack{\text{symmetric sensitivity of } f \text{ to the adjacency entries of } e}} \right) \quad (11)$$

- *No Hessian inverse required* — a single backward pass through the GNN.
- Captures how e reroutes messages between u and v , holding θ fixed.

Two Pathologies of Message Passing

Stacking more GNN layers trades one failure mode for another.

- **Over-smoothing** (Li et al., 2018): as messages propagate, node representations become *indistinguishable* — every node converges to the same embedding.

Two Pathologies of Message Passing

Stacking more GNN layers trades one failure mode for another.

- **Over-smoothing** (Li et al., 2018): as messages propagate, node representations become *indistinguishable* — every node converges to the same embedding.
- **Over-squashing** (Alon and Yahav, 2021): information from distant nodes must be compressed through *narrow bottlenecks* — long-range dependencies get lost.

Two Pathologies of Message Passing

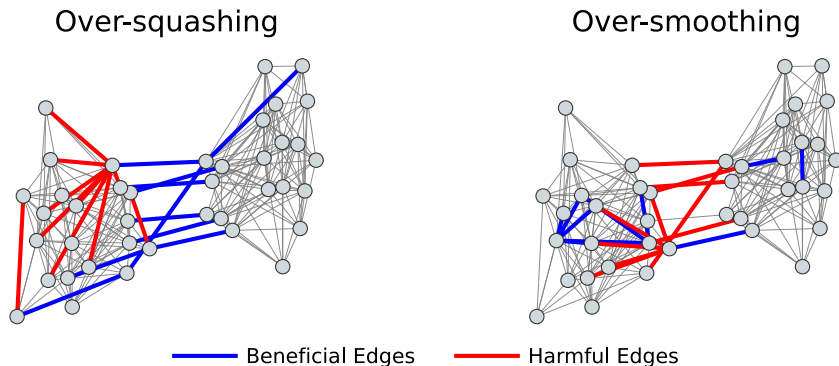
Stacking more GNN layers trades one failure mode for another.

- **Over-smoothing** (Li et al., 2018): as messages propagate, node representations become *indistinguishable* — every node converges to the same embedding.
- **Over-squashing** (Alon and Yahav, 2021): information from distant nodes must be compressed through *narrow bottlenecks* — long-range dependencies get lost.

The dilemma: adding edges between communities widens bottlenecks (helps over-squashing) but also mixes their representations (worsens over-smoothing).

⇒ Whether a specific edge is beneficial or harmful is *not obvious from topology alone*.

Edge Influence on a Barbell Graph

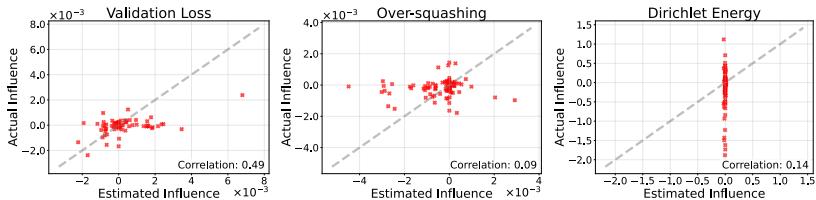


Blue: beneficial edges, Red: harmful edges.

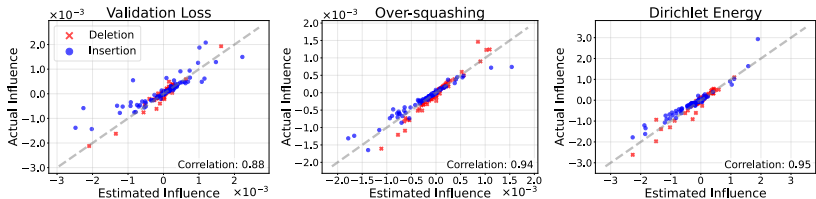
Edges connecting clusters mitigate over-squashing but amplify over-smoothing.

Heo et al. (2025)

Influence Prediction on Three Evaluation Functions f



(a) GIF [7, 35]



(b) Ours

Both components are essential (Heo et al., 2025)

Adversarial Edge Insertion Attack

Attack ratio	Cora (81.0)			CiteSeer (69.3)			PubMed (75.6)		
	1%	3%	5%	1%	3%	5%	1%	3%	5%
DICE	80.8 $\pm$ 0.4	80.3 $\pm$ 0.5	80.0 $\pm$ 0.5	69.0 $\pm$ 0.7	68.5 $\pm$ 0.5	68.1 $\pm$ 0.5	75.1 $\pm$ 0.9	74.5 $\pm$ 1.4	73.5 $\pm$ 1.1
PRBCD	80.6 $\pm$ 0.5	79.4 $\pm$ 0.5	78.7 $\pm$ 0.9	68.5 $\pm$ 0.5	67.4 $\pm$ 0.8	66.4 $\pm$ 0.5	74.6 $\pm$ 1.3	72.9 $\pm$ 1.8	70.9 $\pm$ 1.7
GIF	80.8 $\pm$ 0.4	80.8 $\pm$ 0.6	80.6 $\pm$ 0.4	69.8 $\pm$ 0.7	69.8 $\pm$ 0.7	69.3 $\pm$ 0.5	75.4 $\pm$ 1.1	75.3 $\pm$ 1.0	75.2 $\pm$ 0.9
Ours (DE)	81.0 $\pm$ 0.3	80.7 $\pm$ 0.3	80.8 $\pm$ 0.3	69.1 $\pm$ 0.5	69.2 $\pm$ 0.7	68.8 $\pm$ 0.8	75.6 $\pm$ 1.0	75.2 $\pm$ 1.0	75.0 $\pm$ 1.2
Ours (f_{OQ})	80.9 $\pm$ 0.4	80.9 $\pm$ 0.6	80.6 $\pm$ 0.7	68.9 $\pm$ 0.6	69.0 $\pm$ 0.5	68.6 $\pm$ 0.8	75.4 $\pm$ 0.9	74.9 $\pm$ 1.0	74.4 $\pm$ 1.1
Ours (VL)	80.2$\pm$0.7	79.1$\pm$0.7	78.4$\pm$0.8	68.2$\pm$0.5	66.7$\pm$0.6	65.1$\pm$0.9	73.2$\pm$1.4	71.2$\pm$1.5	69.9$\pm$1.1

identifying the single edge whose addition most degrades accuracy (Heo et al., 2025)

Potential Applications

1. Adversarial Attack & Defense

- Craft structural attacks by targeting the most harmful edges.
- Detect and prune adversarially inserted edges.

2. Principled Graph Rewiring

- Evaluate whether a proposed rewiring actually helps the downstream task.
- Design rewirings that balance *over-squashing* and *over-smoothing*.

3. Graph Data Curation

- Identify noisy or mislabeled edges in knowledge graphs and citation networks.
- Select the most informative edges to label under an active-learning budget.

4. Interpretability & Fairness

- Explain a GNN's prediction on a node by tracing it back to influential edges.
- Audit which relationships drive potentially biased predictions.

Part 3: Group Influence and Data Interactions

“Which data complement each other, and which interfere?”

Work in progress



Jaeseung Heo



Kyeongheung Yun



Youngbin Choi



Sehyun Hwang



Jungseul Ok

Limitation of First-Order Influence

Let S be a subset of data points $\{z_1, \dots, z_m\}$, $g_f = \nabla_{\theta} f(\theta^*)$.

First-order influence is *linear* in the selected subset:

$$f(\theta_{\varepsilon}) - f(\theta^*) \approx -\varepsilon g_f^{\top} H^{-1} g_S, \quad (12)$$

where $g_S = \sum_{i \in S} \nabla_{\theta} L(z_i, \theta^*)$, $H = \nabla_{\theta}^2 R(\theta^*)$.

Limitation of First-Order Influence

Let S be a subset of data points $\{z_1, \dots, z_m\}$, $g_f = \nabla_{\theta} f(\theta^*)$.

First-order influence is *linear* in the selected subset:

$$f(\theta_{\varepsilon}) - f(\theta^*) \approx -\varepsilon g_f^{\top} H^{-1} g_S, \quad (12)$$

where $g_S = \sum_{i \in S} \nabla_{\theta} L(z_i, \theta^*)$, $H = \nabla_{\theta}^2 R(\theta^*)$.

Problem: influence of a group S is just the *sum of individual influences* (Koh et al., 2019):

$$\mathcal{I}(S) = \sum_{i \in S} \mathcal{I}(z_i)$$

Limitation of First-Order Influence

Let S be a subset of data points $\{z_1, \dots, z_m\}$, $g_f = \nabla_{\theta} f(\theta^*)$.

First-order influence is *linear* in the selected subset:

$$f(\theta_{\varepsilon}) - f(\theta^*) \approx -\varepsilon g_f^{\top} H^{-1} g_S, \quad (12)$$

where $g_S = \sum_{i \in S} \nabla_{\theta} L(z_i, \theta^*)$, $H = \nabla_{\theta}^2 R(\theta^*)$.

Problem: influence of a group S is just the *sum of individual influences* (Koh et al., 2019):

$$\mathcal{I}(S) = \sum_{i \in S} \mathcal{I}(z_i)$$

Cannot capture:

- **Redundancy:** two similar points provide diminishing returns
- **Complementarity:** two points together help more than either alone
- **Interference:** two points together hurt despite each being helpful

Second-Order Group Influence

Let $\bar{v} = H^{-1}g_S$, $u = H^{-1}g_f$, $H_f = \nabla_{\theta}^2 f(\theta^*)$, $H_S = \nabla_{\theta}^2 \mathcal{L}_S(\theta^*)$.

Include the ε^2 term to capture *interactions*:

$$f(\theta_{\varepsilon}) - f(\theta^*) \approx \underbrace{-\varepsilon g_f^{\top} \bar{v}}_{\mathcal{I}_1: \text{first-order}} + \frac{\varepsilon^2}{2} \left[\underbrace{\bar{v}^{\top} H_f \bar{v}}_{\mathcal{T}_1: \text{interaction attribution}} + \underbrace{2 u^{\top} H_S \bar{v}}_{\mathcal{T}_2: \text{curvature correction}} \right] \quad (13)$$

- $\mathcal{T}_1$: how the parameter shift interacts with *evaluation* curvature
- $\mathcal{T}_2$: how the group's *training* curvature deflects the update

$\mathcal{T}_1$: Pairwise Interaction Expansion

Let $v_i = H^{-1} \nabla_{\theta} L(z_i, \theta^*)$. Then $\mathcal{T}_1$ decomposes into pairwise terms:

$$\mathcal{T}_1 = \frac{1}{|S|^2} \sum_{i,j \in S} v_i^{\top} H_f v_j \quad (14)$$

Each $v_i^{\top} H_f v_j$ measures how data points i and j *interact* under the evaluation curvature.

$\mathcal{T}_1$: Pairwise Interaction Expansion

Let $v_i = H^{-1} \nabla_{\theta} L(z_i, \theta^*)$. Then $\mathcal{T}_1$ decomposes into pairwise terms:

$$\mathcal{T}_1 = \frac{1}{|S|^2} \sum_{i,j \in S} v_i^{\top} H_f v_j \quad (14)$$

Each $v_i^{\top} H_f v_j$ measures how data points i and j *interact* under the evaluation curvature.

This enables:

- Detecting **redundancy**: $v_i^{\top} H_f v_j > 0$ (diminishing returns)
- Detecting **complementarity**: $v_i^{\top} H_f v_j < 0$ (synergy)

$\mathcal{T}_2$: Curvature Correction (Basu et al., 2020)

$$\mathcal{T}_2 = 2 u^\top H_S \bar{v}, \quad u = H^{-1} g_f, \quad \bar{v} = H^{-1} g_S$$

$\mathcal{T}_2$ measures the *alignment* between the evaluation gradient g_f and the group training gradient g_S , weighted by the group's curvature H_S (preconditioned by H^{-1}).

Application: Selecting Synthetic Data for LLM SFT

Task: select complementary synthetic data to add on top of real data.

Synthetic instruction/solution generation is now standard for LLM fine-tuning (Wang et al., 2023), (Xu et al., 2024), (Yu et al., 2024), but naïve scaling yields *redundant examples*.

Setup:

- Model: Llama-3.1-8B
- Pool: MetaMathQA ($\approx 395\text{K}$ samples: GSM8K + MATH augmented)
- Target: GSM8K
- Warmup on random 5%; select $K = 19,750$ samples for SFT
- 5 epochs SFT, 5 runs

Greedy Data Selection

Let $w_i = H_f v_i$. Predicted gain when adding point j to current set $\mathcal{S}$:

$$\Delta f(j) = \underbrace{-\frac{1}{N} g_f^\top v_j}_{\text{first-order}} + \underbrace{\frac{1}{2N^2} \left(2 \sum_{i \in \mathcal{S}} w_j^\top v_i + w_j^\top v_j \right)}_{\text{interaction}} \quad (15)$$

- Precompute v_i, w_i for all pool data
- Each greedy step reduces to dot products
- Curvature correction term $\mathcal{T}_2$ cannot be added due to complexity.

Results: 5% Selection on MetaMath $\rightarrow$ GSM8K

Method	GSM8K Acc (%)	Δ vs Random
Random	57.91 ± 0.37	—
$\mathcal{I}_1$ top- K	56.21 ± 0.36	-1.70
<i>Greedy</i> ($\mathcal{I}_1 + \mathcal{T}_1$)	58.79 ± 0.42	+0.88

Results: 5% Selection on MetaMath $\rightarrow$ GSM8K

Method	GSM8K Acc (%)	Δ vs Random
Random	57.91 ± 0.37	—
$\mathcal{I}_1$ top- K	56.21 ± 0.36	-1.70
<i>Greedy ($\mathcal{I}_1 + \mathcal{T}_1$)</i>	58.79 ± 0.42	+0.88

- Greedy with the $\mathcal{T}_1$ interaction term *beats random* by +0.88%p.
- First-order top- K is *worse than random* — due to redundancy in the top-ranked samples.

Why Does Greedy Beat $\mathcal{I}_1$ Top- K ?

Selection	GSM (%)	MATH (%)
Full pool (reference)	60.76	39.24
Random	60.78	39.22
$\mathcal{I}_1$ top- K	33.95	66.05
<i>Greedy</i>	82.24	17.76

Why Does Greedy Beat $\mathcal{I}_1$ Top- K ?

Selection	GSM (%)	MATH (%)
Full pool (reference)	60.76	39.24
Random	60.78	39.22
$\mathcal{I}_1$ top- K	33.95	66.05
<i>Greedy</i>	82.24	17.76

- *Greedy selects 82% GSM samples* — *aligned* with the GSM8K target.
- $\mathcal{I}_1$ top- K selects *66% MATH samples* — *misaligned* with the target.

Summary

- **Part 1:** Influence functions approximate LOO retraining
- **Part 2:** Edge influence in GNNs decomposes into parameter shift + message propagation
- **Part 3:** Second-order group influence captures data interactions

Move beyond i.i.d. first-order influence to capture *structure* and *interactions* in training data.

Machine Learning Lab @ POSTECH

ml.postech.ac.kr

A research group at POSTECH working on the theory and practice of learning from structured and limited data.

Research directions:

- Influence functions, data attribution, and data-centric ML
- Graph neural networks and structured prediction
- Generative models and probabilistic inference
- LLM post-training: data selection, alignment, reasoning
- Trustworthy ML: robustness, uncertainty, privacy



Based in Pohang, South Korea.
Part of the CSE Department and
the Graduate School of AI.

Visiting ANU — Looking for Collaborators

I am on *sabbatical at ANU* through early 2027, hosted by the School of Computing.

Two visiting PhD students joining me until mid-June:



MoonJeong Park

GNNs: over-smoothing, generalization, gradient dynamics on graphs



Saemi Moon

machine unlearning for generative models; evaluation benchmarks

Happy to chat after the talk or over coffee — dongwoo.kim@postech.ac.kr

References I

-  Alon, Uri and Eran Yahav (2021). “On the Bottleneck of Graph Neural Networks and its Practical Implications”. In: *International Conference on Learning Representations (ICLR)*.
-  Bae, Juhan et al. (2022). “If Influence Functions are the Answer, Then What is the Question?” In: *Advances in Neural Information Processing Systems (NeurIPS)*.
-  Basu, Samyadeep, Philip Pope, and Soheil Feizi (2021). “Influence Functions in Deep Learning Are Fragile”. In: *International Conference on Learning Representations (ICLR)*.
-  Basu, Samyadeep, Xuchen You, and Soheil Feizi (2020). “On Second-Order Group Influence Functions for Black-Box Predictions”. In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*.
-  Grosse, Roger et al. (2023). “Studying Large Language Model Generalization with Influence Functions”. In: *arXiv preprint arXiv:2308.03296*.

References II

-  Heo, Jaeseung et al. (2025). “Influence Functions for Edge Edits in Non-Convex Graph Neural Networks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*.
-  Jia, Ruoxi et al. (2019). “Towards efficient data valuation based on the shapley value”. In: *The 22nd international conference on artificial intelligence and statistics*. PMLR, pp. 1167–1176.
-  Koh, Pang Wei and Percy Liang (2017). “Understanding Black-box Predictions via Influence Functions”. In: *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pp. 1885–1894.
-  Koh, Pang Wei W et al. (2019). “On the accuracy of influence functions for measuring group effects”. In: *Advances in neural information processing systems* 32.
-  Li, Qimai, Zhichao Han, and Xiao-Ming Wu (2018). “Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning”. In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*.

References III

-  Wang, Yizhong et al. (2023). “Self-Instruct: Aligning Language Models with Self-Generated Instructions”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*.
-  Xu, Can et al. (2024). “WizardLM: Empowering Large Pre-Trained Language Models to Follow Complex Instructions”. In: *International Conference on Learning Representations (ICLR)*.
-  Yu, Longhui et al. (2024). “MetaMath: Bootstrap Your Own Mathematical Questions for Large Language Models”. In: *International Conference on Learning Representations (ICLR)*.